

GENOSCALE: Heterogeneous Cluster Sizing for Genomic Workflows

ANONYMOUS AUTHOR(S)

Genomic workflows are computationally intensive and increasingly run on heterogeneous computing infrastructure with diverse CPU, memory, and cost options. In practice, researchers and workflow developers manually configure clusters based on rough estimates. However, this approach often leads to out-of-memory failures or wasted resources from over-provisioning. This problem couples three challenges that cannot be solved in isolation: predicting per-task memory under high variability, estimating execution times across heterogeneous machines without full profiling, and efficiently searching an exponentially large space of cluster configurations under cost and deadline constraints.

We present GENOSCALE, an end-to-end system for heterogeneous cluster sizing of genomic workflows. GENOSCALE combines (1) FAMR, a memory recommender using per-tool memory consumption distributions to balance OOM risk and waste; (2) PTS-Hybrid, a microbenchmark-driven execution time predictor for heterogeneous machines; and (3) a cluster sizing optimizer combining branch-and-bound with Bayesian optimization, guided by a workflow simulator that quickly evaluates configurations rather than running them on the actual cluster. Across 4,000 task executions from 850 genomic workflows, GENOSCALE reduces memory retry time by up to 96%, memory waste by up to 37%, and execution time prediction error by 34.1% over state-of-the-art baselines. The cluster sizing optimizer reduces cost by 5.5% over pure Bayesian optimization. On 9 bare-metal cluster configurations, GENOSCALE's workflow simulator shows a 3.9% median error against actual executions.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Applied computing** → **Bioinformatics**; • **Computing methodologies** → **Distributed computing methodologies**.

Additional Key Words and Phrases: Heterogeneous cluster sizing, Resource allocation, Genomic workflows

1 Introduction

Genomic workflows are computationally intensive, and modern genomic pipelines process gigabytes to terabytes of sequencing data through DAGs (Directed Acyclic Graphs) comprising tens of interdependent tools [33]. The cost of sequencing has fallen faster than Moore's Law [62], driving exponential increases in genomic data volume. As a result, computational costs are the largest, or one of the largest components, of research budgets [58], making efficient cluster resource management critical. To meet these demands, researchers rely on shared cluster infrastructure, including on-premises HPC, shared research facilities (e.g., Chameleon [29], CloudLab [17]), and, increasingly, cloud platforms composed of heterogeneous machine types. However, effective cluster provisioning is difficult in practice, and poor choices commonly lead to job failures or wasted resources [9, 29].

Executing genomic workflows on heterogeneous cluster infrastructure requires three coupled decisions: 1) *which subset of machine types to provision from the available pool*, 2) *how much memory to assign each task*, and 3) *how to schedule tasks across heterogeneous nodes*. These decisions are interdependent, yet current practices address them independently through ad-hoc approaches based on trial-and-error [9, 23, 63]. As a result, researchers often initially under-allocate resources based on rough estimates, leading to out-of-memory (OOM) failures that terminate jobs and waste all prior computation [7, 8, 63]. After repeated failures, they shift to over-provisioning, which reduces failures but wastes budget and computing resources.

Table 1 shows why uniform allocation policies are insufficient. We apply four uniform memory allocation policies (Static-32GB, Tool-Max, 2×-Mean, and P95) to memory traces from 8,000 executed tasks, and we compute memory waste and OOM failure rates when applying each policy. Static-32GB allocation wastes 86.3% of memory while causing 256 OOM failures. Tool-Max eliminates

Table 1. **Uniform memory allocation policies on 8,000 genomic task executions, including standalone tool runs (§2).** *Static-32GB: a fixed 32 GB per task; Tool-Max: tool's historical maximum memory; 2×-Mean: twice tool's mean; P95: tool's 95th percentile; Wasted (MEM): absolute memory wasted; Wasted (%): the fraction of total allocation unused.*

Allocation Policy	Wasted (MEM)	Wasted (%)	OOM Failure # (%)
Static-32GB	101.7 TB	86.3%	256 (3.3%)
Tool-Max	21.2 TB	51.8%	0 (0.0%)
2×-Mean	19.8 TB	50.1%	136 (1.7%)
P95	5.7 TB	22.3%	384 (4.8%)

failures but wastes 51.8%. 2×-Mean wastes 50.1% with 136 failures, and P95 reduces waste to 22.3% but causes the most failures (384). In DAG workflows, each OOM failure terminates a task mid-execution and discards all prior computation. Even a few failures can therefore waste hours of compute. We observe that *no single-estimate-based policy achieves both low waste and a low failure rate, because each tool exhibits a distinct memory-consumption pattern*. A uniform policy like P95 may suffice for tools with narrow distributions, but for tools like picard-markduplicate (PMd) [12], where most executions require under 32 GB but occasional runs peak at over 100 GB, the same threshold is insufficient.

Memory allocation is just one decision. Automated heterogeneous cluster sizing requires addressing three difficult challenges. First, per-task memory requirements are difficult to predict across diverse genomic inputs. Our characterization (§2) shows that PMd consumed between 1.1 GB and 101.6 GB across our workload, with 2–3× variation even for inputs of similar size due to input-specific properties (read depth, duplication, genomic complexity). Memory prediction must therefore capture both per-tool memory scaling and variation, to balance under-allocation risk against over-allocation waste.

Second, execution times vary across heterogeneous processors in tool-specific ways. Our characterization (§2) shows that picard-collect-wgs-metrics (PCWM, a quality assessment tool) [12] exhibits 40% variation between AMD EPYC and Intel Xeon, while burrows-wheeler-aligner (BWA-MEM, an alignment tool) [34] shows consistent performance across different CPUs. Since this sensitivity varies tool by tool, accurate prediction requires per-tool performance data on every target machine, but full workflow profiling takes hours or days per machine type, multiplied across every machine type in a heterogeneous pool. Execution time prediction must therefore estimate execution time on each machine without running the full workflow on it.

Third, finding the right subset of machines is difficult because the number of possible configurations grows exponentially with the pool size. For just 30 machine types, 2^{30} (over one billion) configurations are possible. Evaluating every configuration is *infeasible*, and per-machine profiling alone cannot predict cluster-level performance because tools interact across heterogeneous nodes. DAG dependencies and cost/deadline trade-offs further complicate this challenge. For example, a configuration that minimizes cost may miss the deadline while one that meets the deadline may be too expensive. Cluster sizing must therefore prune this space efficiently to find configurations meeting both constraints.

Despite extensive work on benchmarking [2, 21, 22, 36, 53], resource prediction [6–8, 63], and cluster optimization [3, 5, 10, 11, 26, 30, 37, 39, 49, 61, 64], researchers still rely on ad-hoc decisions. Benchmarking tools measure the performance of individual machines, but often fail to explain how tools interact across heterogeneous nodes in a workflow as a whole. Resource predictors can produce point estimates for individual tasks, often missing the distributional/dynamic nature of memory consumption that can drive OOM failures. Existing cluster sizing systems typically target

homogeneous clusters with uniform machine types. Heterogeneous cluster sizing for genomic workflows has not been addressed as an end-to-end problem. Failure-aware per-task prediction, execution-time prediction across heterogeneous machines, and configuration search must be solved jointly with a DAG-aware execution model.

We present GENOSCALE, a system that automates heterogeneous cluster sizing for genomic workflows. GENOSCALE combines three components that together address the three challenges above. First, FAMR (Failure-Aware Memory Recommender) models memory consumption as per-tool distributions that scale with input size, weighting under-allocation risk against over-allocation waste to balance OOM failures and memory efficiency. Second, PTS-Hybrid predicts execution time on new machines by combining lightweight microbenchmarks [45] with limited reference workflow runs, avoiding exhaustive per-machine benchmarking. Third, a cluster sizing optimizer combines branch-and-bound (BnB) with Bayesian optimization (BO), guided by a workflow execution simulator that evaluates configurations in seconds rather than running them on a real cluster. BnB narrows the search space and BO explores the top candidates further, finding cost-efficient configurations under cost and deadline constraints.

Our evaluation uses 4,000 task executions from 850 full genomic workflows on Chameleon Cloud [29] over two weeks. FAMR reduces memory waste by up to 37% and retry time by up to 96% over SOTA memory predictors [8, 63]. PTS-Hybrid reduces prediction error by 34.1% in aggregate over SOTA execution time predictors [6, 14], with the lowest prediction error on 8 of 10 tools and accuracy varying by only 3 pp across three heterogeneous processors. The cluster sizing optimizer reduces cost by 5.5% over pure BO, with 57% lower cost variability across 30 trials. On 9 bare-metal cluster configurations, GENOSCALE's predictions match actual execution within 3.9% median and 8.1% worst-case error, compared to 20.2–27.9% for a baseline combining existing predictors.

Our contributions. This work makes the following research contributions.

- (1) A characterization of 8,000 genomic task executions on heterogeneous processors, showing that memory consumption varies by 2–3× even across inputs of similar size and that the sensitivity of execution time to the processor differs across tools (§2).
- (2) **GENOSCALE**, an end-to-end system that frames heterogeneous cluster sizing as an integrated problem of failure-aware prediction, execution-time prediction across heterogeneous machines, and configuration search under DAG-aware execution (§3).
- (3) **FAMR**, a failure-aware memory recommender that models memory as per-tool distributions that scale with input size rather than point estimates, capturing peak timing and per-tool variability (§4.1).
- (4) **PTS-Hybrid**, a microbenchmark-driven execution time predictor that estimates runtime on new machines in a heterogeneous pool without full workflow benchmarking (§4.2).
- (5) A **cluster sizing optimizer** that combines BnB with BO, guided by a workflow simulator that evaluates configurations in seconds or minutes rather than on the cluster (§5).
- (6) A **comprehensive evaluation** of GENOSCALE on Chameleon Cloud across 4,000 task executions from 850 workflows and 9 cluster configurations, validating both simulator and bare-metal performance (§7).

2 Workload Characterization

Genomic workflows compose dozens of specialized tools (e.g., aligners, variant callers, quality checkers) into multi-stage pipelines [33, 40, 57]. Each genomic tool shows distinct computational profiles and resource demands. To understand how genomic tools behave on heterogeneous hardware and how resource allocation policies affect workflow-level executions, we conduct a characterization study on real cluster infrastructure. This characterization uses the GATK best-practices DNA-seq

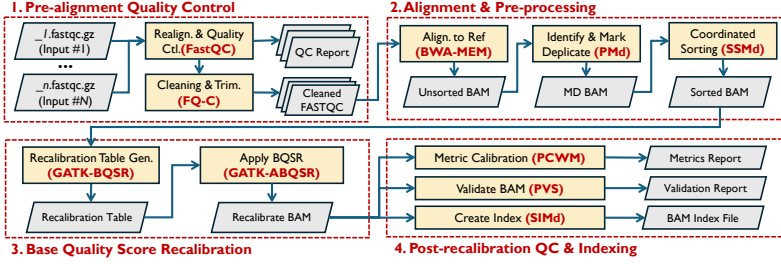


Fig. 1. GATK best-practices DNA-Seq Workflow with four stages. ‘Rectangles’: genomics/bioinformatics tools, ‘Parallelograms’: data or reports.

pipeline [60], a widely adopted genomic analysis workflow comprising 10 genomic tools (Fig. 1). The pipeline begins with quality control via FastQC and fastq-cleaner (FQ-C), proceeds to alignment with BWA-MEM and duplicate marking with PMd and samtools-sort-markduplicate (SSMd), then performs base quality recalibration via GATK BaseRecalibrator (BQSR) and ApplyBQSR (ABQSR), and finishes with validation and indexing through picard-validate-sam (PVS), PCWM, and samtools-index-markduplicate (SIMd). We run this pipeline on Chameleon Cloud across AMD EPYC and Intel Xeon configurations, collecting per-tool metrics at one-second granularity via pidstat [54] across 8,000 task executions, including standalone tool runs and 4,000 tasks executed within 850 end-to-end genomic workflows. Input size refers to the raw reads and excludes the reference genome that each run also reads (5 GB, or 27 GB for the larger runs). Raw inputs range from 100 MB to 6.6 GB, except for a few workflows with 20 GB raw inputs. Analyzing these execution results, we identify two findings that reveal previously unstudied properties of genomic workflows on heterogeneous hardware and motivate GENOSCALE’s design.

Finding 1: Memory consumption variability cannot be explained by input size alone.

Common practice in resource allocation for workflows assumes that memory consumption scales predictably with input size [7, 10, 44, 51]. Our characterization reveals that this assumption may not be valid. Fig. 2 shows peak memory distributions for FastQC across input sizes on identical hardware. Under identical hardware and tool configurations, memory consumption varies by 2–3× across different files of the same input size. Other tools exhibit even more extreme variability. As reported in §1, Picard-markduplicate ranges from 1.1 GB to 101.6 GB across executions, a range that file size alone cannot explain. This variability is driven by input-specific properties of the input data (e.g., read depth, duplication rate, and genomic complexity), which file size does not capture.

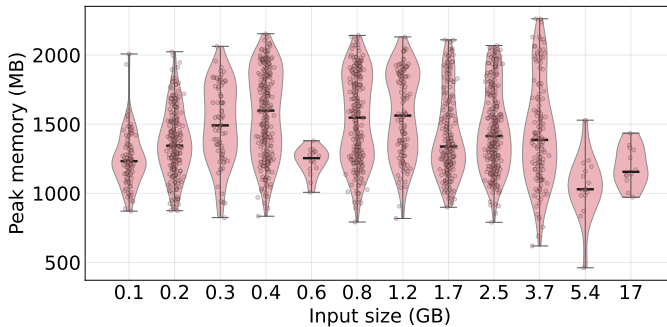


Fig. 2. Memory variability for FastQC across input sizes. Peak memory varies by 2–3× across files of the same input size despite identical hardware.

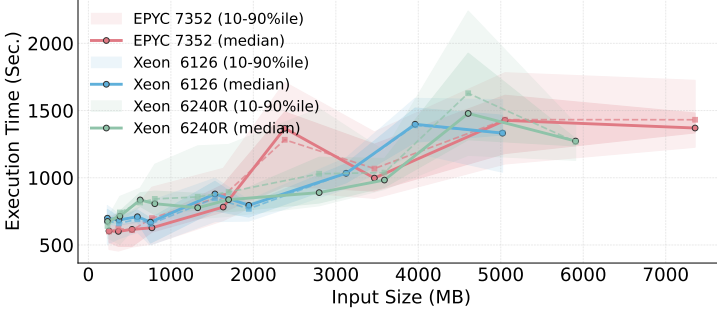


Fig. 3. **Execution time variation for PCWM across processors.** PCWM shows up to 40% execution time variation between AMD EPYC and Intel Xeon, particularly for larger inputs. BWA-MEM, by contrast, shows consistent performance across all three CPUs (BWA-MEM results omitted for space).

Finding ②: Performance variation across heterogeneous processors differs significantly across tools. As workflow execution environments increasingly offer *heterogeneous* processors, accurate performance prediction across these processors becomes essential for resource allocation. We reveal that this variation is not uniform across tools. BWA-MEM runs with consistent execution times across AMD EPYC and two Intel Xeon processors, with its 10–90th percentile bands overlapping at every input size. In contrast, PCWM shows up to 40% variation in execution time across the same three processors (even though all share the x86 architecture), particularly for larger inputs (Fig. 3). This per-tool sensitivity suggests that accurate prediction requires per-tool performance data on every target machine, but exhaustive benchmarking across every machine type is *prohibitively expensive*. Such variability matters most along the workflow’s critical path [1], the longest chain of dependent tasks whose delays propagate to total makespan.

3 GENOSCALE System Architecture

Our findings in §2 motivate GENOSCALE’s design with three components: (1) *FAMR* (§4.1), a failure-aware memory recommender that addresses Finding ① by modeling per-tool memory as distributions and weighting allocation by peak timing; (2) *PTS-Hybrid* (§4.2), a microbenchmark-driven execution time predictor that addresses Finding ② by replacing full per-machine profiling with lightweight microbenchmark-based prediction; and (3) a *cluster sizing optimizer* (§5) that addresses the third challenge in §1 by combining BnB and BO with a workflow simulator, which uses FAMR and PTS-Hybrid predictions to estimate each candidate configuration’s cost and makespan.

GENOSCALE and its components run on top of a Kubernetes-based workflow orchestration engine that executes workflows and collects per-task resource metrics. Given a workflow specification and a pool of candidate machine types, GENOSCALE automatically determines which subset to provision and how to schedule tasks on it. Fig. 4 shows GENOSCALE’s two-phase workflow.

3.1 GENOSCALE’s Two-Phase Workflow

GENOSCALE operates in two phases: (1) a *profiling phase* that collects training data and profiles hardware performance, and (2) a *prediction and cluster sizing phase* that predicts per-task resources and selects a cost-efficient cluster size/configurations for genomic workflows.

(1) Profiling Phase. Users submit workflows as YAML specifications along with a pool of N candidate machine types, where the orchestration engine compiles workflows into DAGs. GENOSCALE profiles the candidate machine pool through three steps. First, it runs lightweight microbenchmarks on every machine to characterize hardware. Second, GENOSCALE runs the full workflow *only once*, on the largest machine in the pool, designated as the reference machine. We choose the largest

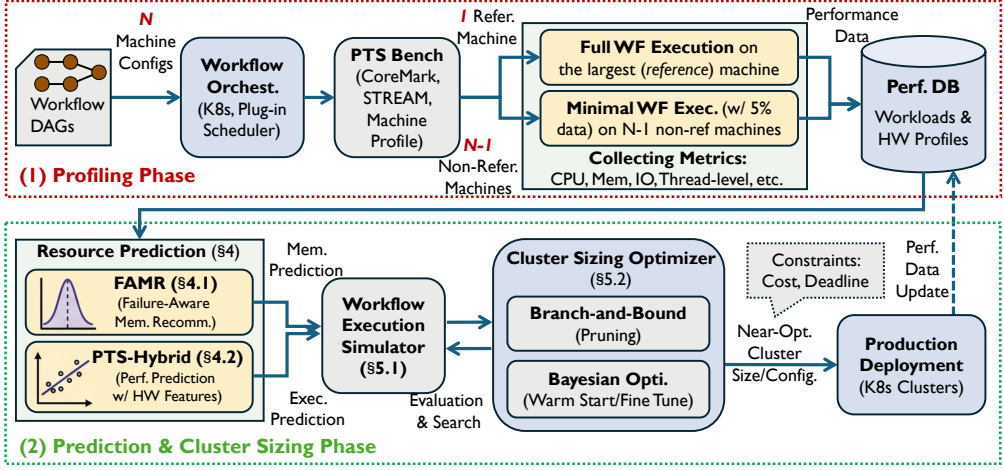


Fig. 4. **GENOSCALE two-phase workflow.** The profiling phase (top) collects per-task and hardware metrics into a performance database; the prediction and cluster sizing phase (bottom) uses FAMR (§4.1), PTS-Hybrid (§4.2), and the cluster sizing optimizer (§5) to search for a cost-efficient configuration under cost and deadline constraints, which GENOSCALE provisions for production execution.

machine as the reference machine because it offers sufficient memory capacity to avoid OOM failures and yields ground-truth per-tool memory and execution traces. Third, it runs the workflow on a 5% input sub-sample on each of the remaining $N-1$ non-reference machines to capture how each tool's behavior varies across hardware. FAMR (§4.1) and PTS-Hybrid (§4.2) derive per-tool memory distributions and execution-time models from these profiles.

Across all three steps, the workflow orchestrator dispatches tasks to a Kubernetes cluster under a user-selected scheduling policy (e.g., HEFT [59], EDF, FIFO, or SJF). The plugin-based scheduler allows the cluster sizing optimizer to evaluate multiple scheduling policies per candidate configuration during simulation. A sidecar running `pidstat` captures per-task CPU, memory, and I/O metrics at one-second granularity (§2), and the orchestrator logs failures (e.g., OOM). All metrics, failure records, and microbenchmark scores are stored in the performance database to train the prediction models (§4) in the next phase.

(2) Prediction and Cluster Sizing Phase. Given a new workflow and a user-provided machine pool, FAMR and PTS-Hybrid (both in §4) predict per-task memory consumption/distribution and the tool's and workflow's execution time from the performance database, respectively. The cluster sizing optimizer (§5) feeds these predictions into a workflow execution simulator that replays the DAG on heterogeneous machines, then combines BnB pruning with BO to converge on a cost-efficient configuration that meets the deadline. GENOSCALE provisions the cluster and configures the orchestration engine with the recommended scheduling policy for production execution.

4 Resource Prediction

Cluster sizing requires *two estimates* for each workflow task: 1) *how much memory to allocate*, and 2) *how long the task will run on a given machine type*. These two prediction problems differ fundamentally. Memory consumption varies widely *within* a tool across executions (§2, Finding 1), so the prediction target is a distribution rather than a point estimate. This motivates our design of FAMR (§4.1), which models per-tool memory distributions and balances under-allocation risk against over-allocation waste.

On the other hand, workflow execution time varies *across* machine types in tool-specific ways (§2, Finding ②), and full benchmarking on every target machine is prohibitively expensive. This motivates our design of PTS-Hybrid (§4.2), which extrapolates execution time from lightweight microbenchmarks. Both predictors update incrementally as new execution data is available.

4.1 FAMR: Failure-Aware Memory Recommender

FAMR models per-tool memory consumption as distributions rather than point estimates. The memory allocation to each genomic tool chooses from its distribution to balance OOM risk against over-provisioning waste.

The key insight is that the right point depends on each genomic tool's variability and peak timing, since OOM failures discard all prior computation while over-provisioning wastes memory. For variable, late-peaking tools, FAMR allocates a larger margin above the predicted peak memory, since OOM at the peak, typically late in execution, discards the most work. For stable, early-peaking tools, FAMR allocates close to the predicted peak memory to avoid memory waste. FAMR proceeds in five steps: characterize the distribution, compute an adaptive allocation, generalize to input sizes outside the observed range, update the model from new observations, and handle OOM failures.

(1) Statistical Characterization. FAMR builds these distributions from the reference machine's full workflow profile (§3.1). Because the (largest) reference machine typically has enough memory capacity to run every tool to completion, FAMR observes each tool's actual peak. A smaller machine would terminate high-memory tools before they reach their peak, causing FAMR to record an underestimated maximum. Memory does not scale linearly with input size, and the scaling differs across tools. For each genomic tool, FAMR therefore models peak memory m as a power law in input size s , fitted by least squares in log space over all observations of the tool:

$$\ln \hat{m}(s) = \beta_0 + \beta \ln s \quad (1)$$

where β is the memory scaling exponent, fitted per tool rather than fixed in advance. FAMR also computes the residual standard deviation σ of the fit and the peak timing ratio $\tau = t_{\text{peak}}/t_{\text{total}}$. Tools that peak late (high τ) lose more work to OOM.

(2) Adaptive Memory Allocation. FAMR implements the above insight by allocating a margin above the predicted peak memory:

$$M_{\text{recom}} = \hat{m}(s) e^{k\sigma} \quad (2)$$

where k determines how much margin the allocation leaves above the predicted peak memory. FAMR chooses k to balance the memory wasted by a larger margin against the expected retry cost of an OOM failure:

$$k^* = \arg \min_k \underbrace{\rho (e^{k\sigma} - 1)}_{\text{waste}} + \underbrace{(1 - \Phi(k)) e^{k\sigma} (\tau + \eta)}_{\text{retry}} \quad (3)$$

where $1 - \Phi(k)$ is the probability that the task's memory demand exceeds its allocation and triggers an OOM failure. η is the cost of re-executing the task, and $\rho \in [0, 1]$ is the fraction of the target node's CPU and memory already reserved by other tasks. Again, the margin is small for stable, early-peaking tools, where OOM failures are relatively cheap. It grows for variable, late-peaking tools (high σ , high τ), where failures are expensive.

(3) Input Size Generalization. Because Eq. 1 is continuous in input size, FAMR can predict memory demand for any target size directly. For sizes outside the observed range, FAMR increases the margin in proportion to the distance of extrapolation, rather than assuming the same confidence it has within the fitted range. The final memory allocation applies a configurable safety margin: $M_{\text{final}} = M_{\text{recom}} \times (1 + \alpha)$, where α trades memory efficiency for failure avoidance.

(4) Incremental Update. After each task completion, FAMR extracts peak memory and time-to-peak from the collected metrics and adds them to the tool's observations, updating Eq. 1 and its residual standard deviation. These updates refine FAMR's per-tool distributions as more executions accumulate, which improves memory allocation accuracy over time.

(5) OOM Failure Handling. Even with a memory margin, an allocation can be insufficient because genomic tools often vary in memory usage (Finding 1). When an OOM failure occurs, FAMR retries the task with a larger memory allocation, scaling it up further on each subsequent failure. Retries stop after a fixed limit (e.g., up to 4 runs), and FAMR reports the task as failed rather than retrying its execution indefinitely. FAMR records the allocation at which the task failed, so that future executions of the same tool receive more memory allocation than that amount.

4.2 PTS-Hybrid: Microbenchmark-Driven Execution Time Predictor

PTS-Hybrid predicts per-tool execution time across heterogeneous machines using lightweight microbenchmarks. Predictions extrapolate from the reference machine and lightweight observations on other machines to a new machine from its microbenchmark scores, avoiding the *prohibitive cost* of full workflow benchmarking on every machine type.

The key insight is that a genomic tool's performance on heterogeneous machines can be predicted from how the tool stresses fundamental hardware resources (CPU cores, memory bandwidth, cache behavior, etc.) measured by standard microbenchmarks. PTS-Hybrid does not assume a specific relationship; however, it learns each tool's dependency on these resources via ridge regression from the profiling measurements ('(1) Profiling Phase' in §3.1). PTS-Hybrid uses Phoronix Test Suite (PTS) [45] as its base benchmark suite, but the predictor architecture is benchmark-agnostic. PTS-Hybrid operates in four steps: characterize each machine via microbenchmarks, learn how each tool's processing rate depends on machine benchmarks, combine benchmark-based and history-based predictions, and incrementally update these tool-specific models from new observations.

(1) Machine Characterization. PTS-Hybrid uses three microbenchmarks on every candidate machine, each chosen to represent a distinct resource demand of genomic tools: CoreMark, Himeno, and STREAM. CoreMark measures integer compute performance (e.g., alignment, sorting), Himeno measures floating-point performance (e.g., quality recalibration), and STREAM quantifies memory bandwidth (e.g., large file scanning). Each machine i is characterized by its scores on the three benchmarks ($\{b_{i,1}, b_{i,2}, b_{i,3}\}$), where $b_{i,j}$ is machine i 's score on benchmark j . Because the three benchmarks report scores in different units and scales (e.g., iterations/s vs. MB/s), scores are normalized to $[0, 1]$ via max-normalization across machines: $\hat{b}_{i,j} = \frac{b_{i,j}}{\max_i(b_{i,j})}$. Additional benchmarks (e.g., disk I/O) can be added by extending the score vector without architectural changes.

(2) Processing Rate Model. For a tool T on machine M , PTS-Hybrid models the processing rate (MB/s) as a linear function of M 's normalized benchmark scores:

$$R(T, M) = \beta_0 + \sum_{j=1}^m \beta_j \hat{b}_{M,j} \quad (4)$$

where $\hat{b}_{M,j}$ is M 's normalized score on benchmark j , and the tool-specific coefficients β_j are learned via ridge regression from the profiled machines. We specifically adopt ridge regression because its L_2 regularization can prevent overfitting on the small reference dataset and stabilizes coefficients in the presence of correlated benchmarks. Moreover, each tool's coefficient pattern reflects which hardware characteristics drive its performance. For example, PCWM, which showed 40% variation across heterogeneous processors (Fig. 3 in §2), assigns higher weight to compute-related benchmarks. On the other hand, BWA-MEM, which is consistent across processors, distributes weights more

evenly. Non-negativity constraints on β_j ($j > 0$) ensure that better hardware always implies higher processing rates. Predicted execution time is $E_{pred} = S/R(T, M)$ for input size S .

(3) Hybrid Prediction Strategy. Benchmark-based prediction alone *may miss* tool-specific runtime behaviors not fully captured by microbenchmarks (e.g., I/O patterns). On the other hand, history-based prediction tends to be accurate on previously profiled machines but does not generalize to new ones. PTS-Hybrid combines both sources through confidence-weighted averaging:

$$E_{hybrid} = w_{tool} \cdot E_{tool} + w_{pts} \cdot E_{pts} \quad (5)$$

where E_{tool} is the history-based estimate from past executions of T on M (the reference machine's full workflow execution and minimal workflow runs on non-reference machines, §3.1; scaled to the target input size), E_{pts} is the benchmark-based estimate (Eq. 4), and the weights w_{tool} , w_{pts} favor the source with more observations and lower variance. PTS-Hybrid automatically selects one of three modes based on data availability. (a) *Hybrid mode* uses E_{hybrid} (Eq. 5) when both benchmark scores and execution history are available, the default case under GENOSCALE's profiling workflow. (b) *PTS-only mode* uses E_{pts} alone for machines without execution history (e.g., when added to the pool after minimal profiling and microbenchmarks). (c) *History-only mode* uses E_{tool} alone, which computes processing rate directly from past executions when benchmark scores are missing.

(4) Incremental Update. After each task completion, PTS-Hybrid extracts the observed processing rate and refits the ridge regression coefficients β_j with the expanded dataset.

5 Cluster Sizing Optimization

Resource predictions alone do not solve the cluster sizing problem. Given per-task memory and execution time predictions (§4), GENOSCALE selects a subset of heterogeneous machines from the available pool to execute the workflow, minimizing cost while meeting the deadline. With n machine types in the pool, the optimizer must decide which types to include, yielding 2^n possible subsets. Therefore, exhaustive evaluation is *infeasible*.

GENOSCALE tackles this challenge by reducing the search space through two components. The *workflow execution simulator* (§5.1) serves as an evaluation tool. Given a candidate machine subset and scheduling policy, it replays workflow execution using FAMR and PTS-Hybrid predictions and returns cost and makespan estimates in seconds rather than the hours of real execution. The *cluster sizing optimizer* (§5.2) drives the search by generating candidates and invoking the simulator to evaluate them, combining BnB pruning with BO refinement to converge on a cost-efficient selection under deadline constraints. The optimizer's first step uses fast predictions to filter infeasible candidates without invoking the simulator; subsequent steps invoke the simulator on promising configurations for accurate estimates.

5.1 Workflow Execution Simulator

The simulator must capture two aspects of workflow execution that affect cost and makespan: *task-level dependencies* (a task cannot start until its predecessors complete) and *resource contention* (a task remains pending if no node has sufficient capacity, until other tasks release resources). GENOSCALE uses simulation to capture these dynamics.

Simulation Loop. The simulation proceeds through fixed time steps (e.g., 1 sec.). At each scheduling tick, the simulator dispatches ready tasks (those whose predecessors have completed) under the selected scheduling policy (HEFT, EDF, FIFO, or SJF). For each ready task, the SCHEDULEANDPREDICT (j, F, P, FS, M') function performs four steps. (1) it computes total input size from the file size lookup. (2) it queries FAMR for obtaining the predicted memory and applies safety margins to derive the task's memory request and limit. (3) it attempts to place the task on a feasible node via the scheduler,

Algorithm 1 SIMULATEWORKFLOW(G, S, F, P, M', FS) $\rightarrow$ ($cost, makespan, metrics$)

```

1:  $t \leftarrow 0$ ;  $ready \leftarrow roots(G)$ ;  $running, done \leftarrow \emptyset$ 
2: while  $|done| < |V|$  and  $t < MAX\_TIME$  do
3:   if  $t \bmod SCHED\_INTERVAL = 0$  then
4:     for each  $j \in ready$  in  $S$ -order: SCHEDULEANDPREDICT( $j, F, P, FS, M'$ ) // §5.1
5:   end if
6:   for all  $j \in running$  with  $t \geq j.end$  do
7:     move  $j$  to  $done$ ; RELEASERESOURCES( $j$ ); add ready successors of  $j$ 
8:   end for
9:    $t \leftarrow t + TIME\_STEP$ 
10: end while
11: return ( $cost, makespan, per\_task\_metrics$ )
  
```

leaving the task in the ready set if no node has sufficient capacity. Finally, (4) it queries PTS-Hybrid for execution time on the assigned node, falling back to a default time perturbed by Gaussian noise if no benchmark or history data exists. When a task completes, the simulator releases its resources and adds any newly-ready successors to the ready set. The loop terminates when all tasks complete or a maximum simulated time is reached. Algo. 1 summarizes this loop. The two predictions are therefore coupled within each task: FAMR’s memory recommendation determines which node can host the task, and that node determines the execution time PTS-Hybrid predicts.

Outputs. The simulator generates three outputs to the cluster sizing optimizer (§5.2): cost (per-machine hourly costs summed over the makespan), makespan (estimated workflow execution time on the candidate subset), and per-task metrics with OOM indicators that flag insufficient memory allocations. These outputs allow the optimizer to penalize subsets that would trigger failures.

5.2 Cluster Sizing Optimizer

Heterogeneous machine types differ significantly in cost and performance, complicating cluster sizing decision. For example, cheaper or fewer machines can reduce cost but extend makespan, and faster or more machines can shorten makespan but raise cost. Formally, given a workflow $\mathcal{W}$, a machine pool M , and a deadline T_{dl} , the optimizer solves $\min_{M' \subseteq M, \pi} C(M', \pi, \mathcal{W})$, subject to $T(M', \pi, \mathcal{W}) \leq T_{dl}$, where T is the makespan and C is the total cost, both estimated by the simulator (§5.1) under scheduling policy π on subset M' . The simulator’s OOM indicator filters out subsets whose memory capacity is insufficient for any task. The optimizer outputs a cost-efficient selection (M^*, π^*) .

As exhaustively evaluating all 2^n subsets and policies is *infeasible*, the optimizer reduces this search through three steps (shown in both Algo. 2 and Fig. 5): (1) Pre-Filtering, (2) BnB, and (3) BO refinement, detailed below. Moreover, both (2) BnB and (3) BO refinement steps collaboratively form a warm-start strategy for faster cost-efficient selection. BnB

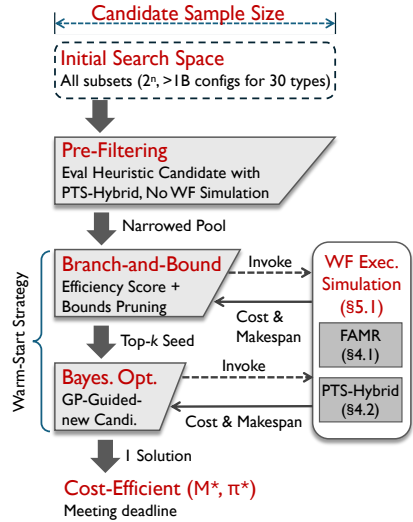


Fig. 5. **GENOSCALE cluster sizing optimizer.** 3-step search over 2^n subsets: ‘Pre-Filtering’, ‘BnB’, and ‘BO refine.’ BnB and BO invoke the simulator (§5.1), forming a warm-start strategy.

narrows the search to low-cost candidates, and BO uses these as starting points instead of random samples, reducing the number of BO iterations.

(1) Pre-Filtering. The optimizer generates a set of initial candidates using heuristic strategies (e.g., greedy selection, cost-efficient, balanced) and evaluates each using PTS-Hybrid predictions and a simple, yet fast greedy heuristic (longest-task-first to the least-loaded machine) rather than running the simulator for all the candidate subsets. After this step, candidates whose estimated makespan exceeds the deadline are removed.

(2) BnB (Branch-and-Bound). We adopt BnB, a classic optimization technique that explores a search tree and prunes subtrees provably worse than the current best. This step first ranks machines by a workload-specific efficiency score combining performance, cost-efficiency, and CPU/memory utilization match:

$$eff_i = w_1 \cdot \frac{perf_i}{p_{norm}} + w_2 \cdot \frac{1}{1 + cost_i/(\beta \cdot perf_i)} + w_3 \cdot util_i \quad (6)$$

where $perf_i$ aggregates PTS-Hybrid scores for machine i , $util_i$ measures how well i 's CPU/memory ratio matches the workload, and $w_1 + w_2 + w_3 = 1$ are user-configurable weights. The optimizer expands a search tree where each level adds one machine type in efficiency order. At each node, a lower bound on cost is computed by fractional knapsack relaxation, allowing fractional machine usage. Nodes are pruned when their lower bound exceeds the current best, or when the partial configuration cannot meet the deadline regardless of further additions. Complete selections are evaluated by the simulator under all scheduling policies $\pi \in \Pi$, and the minimum-cost result updates the current best (M^*, π^*). At each tree depth, only the top BEAM_WIDTH nodes (default 500) are explored further; the rest are discarded.

(3) BO Refinement. This step refines candidates from the previous steps using BO, with the feasible pre-filtering candidates (Step 1) and top BnB configurations (Step 2) as warm-start seeds. BO is a sample-efficient method that fits a probabilistic model to evaluated configurations and uses it to select the next candidate. These candidates have already passed bounds-based pruning and policy simulation, and they provide a high-quality seed dataset rather than random initialization. The candidate subsets are encoded as binary vectors $\mathbf{x} \in \{0, 1\}^n$, where $x_i = 1$ indicates machine M_i is selected. The objective incorporates a soft deadline penalty:

$$f(\mathbf{x}) = C(\mathbf{x}) + \lambda_T \cdot \max(0, T(\mathbf{x}) - D)^2 \quad (7)$$

where λ_T controls the trade-off between cost and deadline satisfaction, with C and T from simulation. A Gaussian process is fit to the seed configurations. The next candidate is then selected by maximizing expected improvement over a search pool that includes previously evaluated configurations, perturbations of the current best (e.g., swapping single machine types), and random samples. This allows BO to explore configurations that BnB may have pruned conservatively. The simulator evaluates each new candidate under the best policy carried over from the warm-start. The loop terminates when improvement plateaus.

Together, the simulator (§5.1) and optimizer (§5.2) find a cost-efficient configuration (M^*, π^*) that meets the deadline.

6 Implementation

Resource Predictors. FAMR maintains per-tool memory observations in SQLite and updates each tool's fit (Eq. 1) and residual standard deviation on each job completion. PTS-Hybrid runs lightweight microbenchmarks on each new machine type and trains a ridge regression (scikit-learn)

Algorithm 2 Heterogeneous Cluster Sizing (BnB + BO)

Input: workflow $\mathcal{W}$, machine pool M , deadline T_{dl} , policies Π
Output: selection (M^*, π^*) meeting deadline

```

1: procedure HETCLUSTERSIZING( $\mathcal{W}, M, T_{dl}, \Pi$ )
2:   // Step 1: Pre-Filtering
3:    $C \leftarrow \{c \in \text{HEURISTICS}(M) : \text{ESTMAKESPAN}(c) \leq T_{dl}\}; M^* \leftarrow \emptyset$ 
4:   // Step 2: Branch-and-Bound
5:   sort  $M$  by efficiency score // Eq. (6)
6:   for each node  $v$  in BnB tree (top BEAM_WIDTH) do
7:     if LOWERBOUND( $v$ )  $\geq \text{cost}(M^*)$  or  $\neg \text{FEASIBLE}(v)$  then prune
8:     else if COMPLETE( $v$ ) then
9:        $y^* \leftarrow \min_{\pi \in \Pi} \text{SIMULATE}(v, \pi, \mathcal{W})$ ; update  $(M^*, \pi^*)$  if  $y^* < \text{cost}(M^*)$  // Algo. 1
10:      else branch on next machine type
11:    end if
12:  end for
13:  // Step 3: BO Refinement
14:   $D \leftarrow (\text{top K_SEED from Step 2}) \cup C$ 
15:  fit Gaussian process on  $D$  with objective  $f$  // Eq. (7)
16:  for  $t = 1$  to BO_ITERS do
17:     $x_t \leftarrow \arg \max_x \alpha_{EI}(x \mid D)$  // expected improve.
18:     $y_t \leftarrow \text{SIMULATE}(x_t, \pi_{x_t}^*, \mathcal{W})$ ;  $D \leftarrow D \cup \{(x_t, y_t)\}$  // Algo. 1
19:    break if no improvement over last LAST_K iters
20:  end for
21:  return  $(M^*, \pi^*)$ 
22: end procedure

```

mapping benchmark scores to per-tool execution times. Predictions are cached per (tool, machine) pair to avoid redundant model queries.

Workflow orchestrator engine and cluster sizing optimizer. The orchestration engine is implemented in Node.js/TypeScript and integrates with Kubernetes through informers for efficient state synchronization rather than polling. The optimizer is implemented in Python using scikit-optimize for BO with Gaussian processes and NumPy for BnB calculations. It also provides an HTTP API accepting machine pools, workflows, and constraints, returning JSON results with the recommended configuration.

Workflow simulator. The simulator uses configurable TIME_STEP (in Algo. 1) of 1 second by default with up to 500 $\times$ acceleration, reducing hours or days of real execution into seconds or minutes of simulation. To achieve this, in-memory providers replace Kubernetes API calls and filesystem accesses, with metadata for genomic datasets preloaded to enable instant file-size lookups during simulation. Additionally, repeated prediction queries from FAMR and PTS-Hybrid are cached and reused, achieving 85%–90% hit rates that reduce per-query latency from 50ms to less than 1ms. Moreover, the simulator can be invoked during both BnB and BO steps, but results are cached to avoid redundant evaluation.

Hyperparameters. We use default values for hyperparameters. *i.e.*, efficiency-score weights $w_1 = w_2 = 0.3$, $w_3 = 0.4$ (higher utilization weight to favor machines whose resource ratio matches the workload), and a beam width of 500 for branch-and-bound. FAMR’s safety margin α (§4.1) is configurable, and we evaluate three settings ($\alpha=0.2, 0.35, 1.0$) in §7.3. The deadline penalty weight λ_T is auto-scaled based on baseline cost. Source code will be released as open source upon publication.

7 Evaluation

We evaluate GENOSCALE to answer the following questions: First, how accurately do FAMR and PTS-Hybrid predict per-task memory and execution time on heterogeneous machines (§7.2, §7.3)? Second, does the cluster sizing optimizer find cost-efficient configurations under deadline constraints, and does the BnB/BO warm-start significantly improve over pure BO (§7.4)? Third, do the simulator-driven decisions translate to real cluster execution? In other words, are GENOSCALE’s predicted makespans consistent with actual execution on heterogeneous bare-metal hardware (§7.5)? Lastly, do FAMR and PTS-Hybrid still provide accurate predictions on larger inputs (§7.6)?

7.1 Experimental Setup

Infrastructure and Heterogeneous Machine Pool. We run experiments on Chameleon Cloud bare-metal across three processor types: AMD EPYC 7352 (96 vCPUs), Intel Xeon Gold 6126 (48 vCPUs), and Intel Xeon Gold 6240R (96 vCPUs), each with 192 GB RAM. We construct a pool of 30 machine configurations (1–64 cores, 4–256 GB RAM) by logically partitioning these nodes into VMs, assigning each configuration a synthetic hourly cost (\$0.08–\$5.12) modeled on public clouds [4, 20]. The resulting 60× cost range with 30 machine types produces 2^{30} (over one billion) candidate configurations. For real-cluster validation, we use 10 separate bare-metal machines, sampling 4 to 7 without replacement to construct each of the 9 evaluation configurations.

Workloads. The workload is the GATK best-practices DNA-seq pipeline (Fig. 1). While §2 analyzes all 8,000 task executions, including standalone tool runs, per-task evaluation (§7.2, §7.3) uses the 4,000 that ran within end-to-end workflows, since they reflect how each tool behaves in a production pipeline. For the optimizer evaluation (§7.4), we submit 200 DNA-seq workflows to GENOSCALE (40 on 2.75 GB inputs, 80 on 850 MB inputs, 80 on 100 MB inputs), under a 600-minute deadline and \$150 cost budget. For real-cluster validation (§7.5), we provision GENOSCALE’s cost-efficient configurations on actual Chameleon bare-metal machines and run 10 parallel DNA-seq workflows on each of 9 configurations.

Baselines. For execution time prediction, we compare against Lotaru [6], the SOTA workflow execution time predictor. Lotaru-A (aggressive bandwidth) and Lotaru-G (gentle bandwidth) apply LOESS (locally estimated scatterplot smoothing) regression, bounding the bias-variance trade-off. OnlineP [14] uses density-based clustering with statistical distribution sampling for uncorrelated data. For memory prediction, we compare against Sizely [8], which trains multiple regression models and selects one per task. We also use three classical statistical baselines. P95 takes the per-tool 95th percentile of historical memory, while WittPer. and WittReg. from Witt *et al.* [63] apply a per-tool percentile with retry feedback and an input-size regression with overprediction offset, respectively. For cluster sizing, we compare GENOSCALE’s BnB+BO optimizer (§5) against BO-Classic, which is a pure BO without the warm-start mechanism.

Performance Metrics. We measure prediction accuracy using RMSE, MAE, and MAPE. For memory allocation, we use OOM failure rate, memory utilization, and retry time, defined as the compute time lost to failed jobs and re-execution. For cluster sizing, we report total cost, makespan, and inter-trial cost variability across 30 independent trials.

7.2 Execution Time Prediction

We evaluate PTS-Hybrid at three scales: (1) per-tool prediction accuracy, (2) prediction across heterogeneous machines, and (3) workflow-level robustness.

Per-Task Prediction Accuracy. Table 2 reports per-tool RMSE for all four predictors across the 10 tools in the workflow. PTS-Hybrid achieves the lowest RMSE on 8 of 10 tools and reduces total

Table 2. **Per-tool RMSE (in sec.) on the GATK DNA-seq pipeline.** ‘Improvement’ is PTS-Hybrid’s RMSE reduction over the lowest-RMSE baseline in each row. Bold marks the best per row.

Tool	Lotaru-A	Lotaru-G	OnlineP	PTS-Hybrid	Improvement
FastQC	154.4	130.2	217.9	53.1	59.2%
FQ-C	2885.4	884.2	970.4	363.1	58.9%
BWA-MEM	1352.0	682.9	602.1	201.0	66.6%
PMd	568.4	426.9	434.8	479.0	-12.2%
SSMd	132.0	45.2	47.7	32.5	28.2%
GATK-BQSR	1191.9	807.4	1278.2	672.6	16.7%
GATK-ABQSR	437.9	260.2	330.0	215.8	17.1%
PVS	187.2	108.9	113.4	58.6	46.2%
PCWM	499.9	216.2	633.9	266.6	-23.3%
SIMd	21.8	18.8	20.0	17.6	6.2%
Total (Sum)	7430.9	3580.8	4648.5	2360.0	34.1%

RMSE by 34.1% compared to the strongest baseline (Lotaru-G, 3580.8s vs. 2360.0s), with the largest improvements on BWA-MEM (66.6%), FastQC (59.2%), FQ-C (58.9%), and PVS (46.2%). MAE and MAPE measurements confirm the same trend (results omitted for space).

Each baseline fails on a different tool class. OnlineP’s density-based clustering of historical runtimes degrades on high-variance tools, resulting in 119% MAPE for FastQC vs. PTS-Hybrid’s 22%. Lotaru-A and Lotaru-G apply LOESS smoothing based solely on input size, producing reasonable predictions for size-correlated tools (e.g., SIMd), but accumulating errors for compute-bound tools (e.g., GATK-BQSR, BWA-MEM) where hardware characteristics dominate. PTS-Hybrid’s hybrid mode (E_{hybrid} in Eq. 5) combines microbenchmark-based and history-based predictions, while its per-tool ridge regression (Eq. 4) models each tool’s sensitivity to CPU compute and memory bandwidth, capturing the execution time variation across heterogeneous processors observed in Finding 2 (in §2). The exception is content-driven variability (e.g., duplication rates). PTS-Hybrid yields 12.2% higher RMSE on PMd and 23.3% higher on PCWM, where Lotaru’s history-based smoothing handles noise more gracefully. Despite these two cases, PTS-Hybrid maintains the lowest aggregated RMSE across the pipeline.

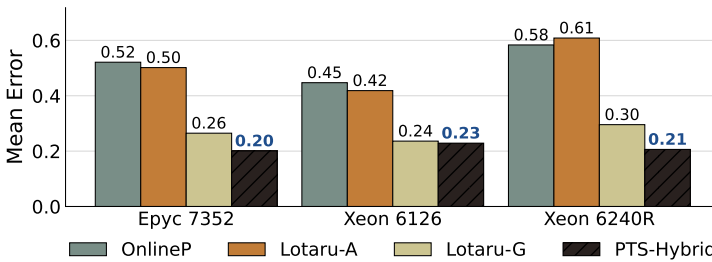


Fig. 6. **MAPE across three heterogeneous x86 processors.** Lower is better.

Prediction across Heterogeneous Machines. Fig. 6 reports MAPE for all four predictors across the three heterogeneous machines. PTS-Hybrid achieves 20–23% MAPE across AMD EPYC 7352, Intel Xeon Gold 6126, and Intel Xeon Gold 6240R, varying by 3 percentage points (pp). Lotaru-G (the strongest baseline in the previous evaluation) performs comparably to PTS-Hybrid on Xeon 6126 (24% vs. 23%) but degrades to 30% on Xeon 6240R, a 6 pp difference from PTS-Hybrid on this machine. Lotaru-A and OnlineP show much larger shifts (42–61% and 45–58%, respectively).

These wide variations of the baselines’ accuracy show why history-based prediction struggles across heterogeneous hardware. These baselines learn how runtime depends on input size. But

runtime also depends on hardware characteristics like CPU speed and memory bandwidth, so this relationship differs from machine to machine. Therefore, the prediction accuracy is likely to drop on machines that differ from the training set. However, PTS-Hybrid avoids this dependency by mapping benchmark scores to processing rates via ridge regression (Eq. 4). A machine's three benchmark scores fully parameterize the prediction, eliminating per-machine retraining. Because the prediction depends on benchmark scores instead of past runtimes, PTS-Hybrid's prediction accuracy (MAPE) varies by only 3 pp across the three machines.

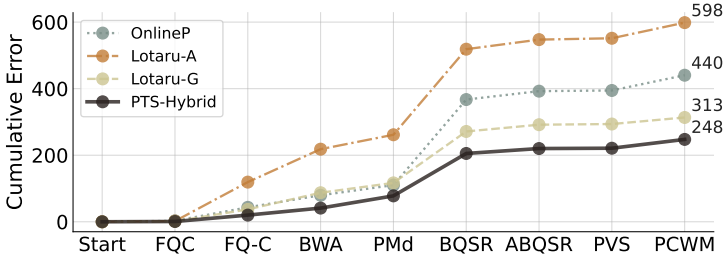


Fig. 7. Cumulative prediction error (with critical path).

Workflow-Level Robustness. Fig. 7 reports cumulative prediction error along the workflow's critical path, where errors directly affect the overall makespan. Two off-critical-path indexing tools (SSMd, SIMd) are omitted. PTS-Hybrid finishes at 247.5s of cumulative error, compared to 313.4s for Lotaru-G, 440.4s for OnlineP, and 598.4s for Lotaru-A. The largest single-stage jump occurs at GATK-BQSR for all four predictors, where the compute-heavy base quality recalibration step accelerates error growth. PTS-Hybrid stays the lowest at this stage too, preventing the spike from propagating to downstream tools through DAG dependencies.

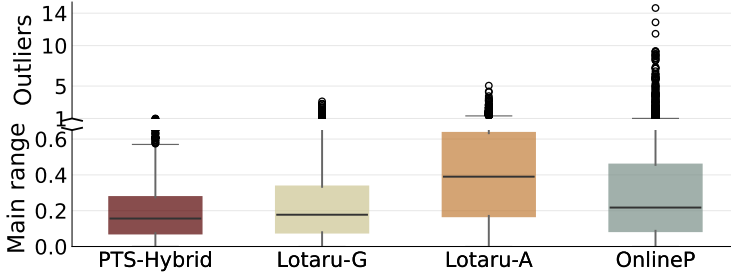


Fig. 8. Box plot of per-task relative prediction errors. Lower panel (0–0.6) shows the main range and whiskers; upper panel (1–14) shows outliers. PTS-Hybrid's outliers stay below 2×, while OnlineP's reach 14×.

Fig. 8 shows the per-task distribution of relative prediction error ($|\text{predicted} - \text{actual}| / \text{actual}$). PTS-Hybrid produces the tightest box, with the middle 50% of predictions within 0.1–0.3 around a median near 0.15, and outliers staying below 2×. Lotaru-G and Lotaru-A produce wider boxes with outliers reaching 3× and 3.5×, respectively. OnlineP's box is wider still, with outliers extending to 14× relative error, indicating frequent catastrophic mispredictions on inputs unlike its training set. PTS-Hybrid is the only predictor whose worst-case error stays bounded within a small range.

7.3 Memory Allocation with FAMR

We evaluate FAMR at both the task and workflow levels: (1) the memory–retry trade-off against SOTA baselines and (2) workflow-level impact through DAG dependencies. We evaluate three FAMR configurations distinguished by the safety margin α (in §4.1), which scales the recommended

allocation by $1+\alpha$. FAMR-A (Aggressive, $\alpha=0.2$) minimizes memory waste, FAMR-C (Conservative, $\alpha=1.0$) doubles the recommendation to maximize failure avoidance, and FAMR-B (Balanced, $\alpha=0.35$) sits between them.

Memory-Retry Trade-off. Memory allocation poses a two-objective trade-off. Under-allocation of memory triggers OOM failures and retries that waste compute time, while over-allocation wastes memory. Fig. 9 shows both axes simultaneously across all methods. FAMR configurations cluster in the lower-left region (low waste, low retry), while Sizely performs worst on both axes with 3.72 TB memory waste and 242K seconds (67 hours) of retry time. FAMR-B outperforms Sizely on both axes. It shows 34% lower waste and 72% lower retry, an improvement on both axes rather than a trade-off between them. Table 3 also reports memory waste and retry time across all seven methods. FAMR-A achieves the lowest waste (2.34 TB) at 93.8K seconds retry, while FAMR-C achieves the lowest retry (9.4K seconds, 26 $\times$ lower than Sizely) at 3.19 TB waste. Moreover, we observe that Sizely’s point estimate often misses memory variability and late-peak timing, while FAMR’s per-tool distribution and timing-weighted margin (Eq. 3) capture both. For PMd, a high-variance tool, Sizely retries on 1 in 6 executions vs. FAMR-C’s 1 in 33.

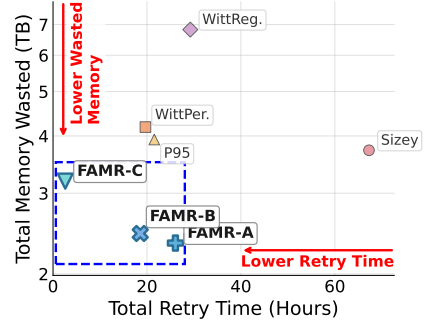


Fig. 9. **Memory wastage vs. retry time across methods.** FAMR variants (ours) cluster in the lower-left region with low wastage and low retry time. Y axis is log-scaled.

Table 3. **Memory waste and retry time across allocation methods.** FAMR-A achieves the lowest waste (2.34 TB), FAMR-C shows the lowest retry (9.4K sec.), and FAMR-B improves on Sizely along both axes. Bold marks the best per column.

Method	Memory Waste	Retry Time	vs. Sizely (waste/retry)
FAMR-A	2.34 TB	93.8K sec.	−37% / −61%
FAMR-B	2.45 TB	66.8K sec.	−34% / −72%
FAMR-C	3.19 TB	9.4K sec.	−14% / −96%
Sizely	3.72 TB	242K sec.	— (baseline)
P95	3.93 TB	77.6K sec.	+6% / −68%
WittPer.	4.18 TB	70.8K sec.	+12% / −71%
WittReg.	6.82 TB	105K sec.	+83% / −57%

Workflow-Level Impact. We measure the workflow-level time loss caused by OOM failures, since downstream tasks must wait for a failed task to be retried and completed. Fig. 10 reports this loss at each pipeline stage, averaged over 850 workflow executions. Specifically, we compare FAMR variants against Sizely, the only baseline that trains models from execution data rather than applying a fixed statistical rule, and therefore the closest competitor for FAMR. The results show that Sizely loses 52.7% of total runtime to failures, indicating that over half of all execution time is wasted. FAMR-C loses only 3.2%, a 16 $\times$ reduction. FAMR-A and FAMR-B lose 22.2% and 17.5%, respectively. Both results are significantly smaller than Sizely’s execution-time waste but higher than FAMR-C’s, which is consistent with their lower safety margins.

The majority of Sizely’s loss is due to the difficulty of predicting genomic tools’ memory demand with a single estimate. For example, Sizely’s largest single-stage loss occurs at PCWM, where Sizely averages 0.32 retries per execution (about 1 retry every 3 runs) while FAMR-C averages 0.02 (1 retry every 50 runs). Each retry adds substantial time to complete the critical path, and these costs

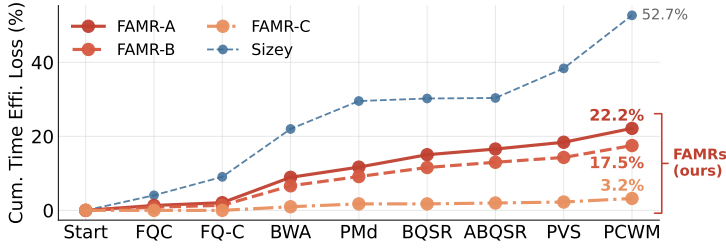


Fig. 10. Time efficiency loss through pipeline. Sizy accumulates 52.7% loss; FAMR-C incurs just 3.2%.

accumulate across the workflow's later stages. FAMR configurations show much smaller loss at the same stages because the timing-weighted margin (Eq. 3) grows large only for high-variance, late-peaking tools, avoiding both under-allocation on risky tools and over-allocation on stable ones.

7.4 Cluster Sizing Optimization

We compare GENOSCALE's BnB+BO optimizer (§5) against BO-Classic across the 30-trial evaluation. Fig. 11(a) shows the cost distribution across trials. BnB+BO's mean cost (\$120.13) is 5.5% lower than BO-Classic's (\$127.08). BnB+BO's cost also varies far less across trials. Its costs between the 25th and 75th percentiles span \$120.4 to \$122.5, compared with \$124.5 to \$129.4 for BO-Classic. Therefore, its interquartile range is 57% narrower (\$2.1 vs. \$4.9). Both differences are statistically significant (t-test and Kolmogorov-Smirnov test, $p < 0.001$). Fig. 11(b) shows the budget-meeting probability. Across the plotted budget range, we observe that BnB+BO meets each budget at least as often as BO-Classic. For example, a \$122 budget is met in 65% of BnB+BO trials but only 5% of BO-Classic trials.

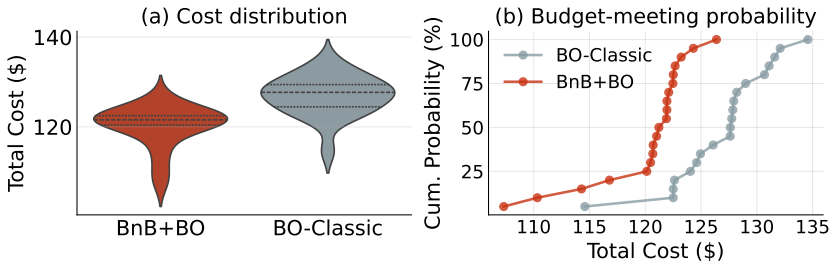


Fig. 11. BnB+BO vs. BO-Classic across 30 trials. (a) Cost distribution and (b) Budget-meeting probability.

BnB+BO also converges faster. It reaches 90% of its final improvement within 10 iterations, while BO-Classic needs 20. Optimization time drops from 12–15 minutes for BO-Classic to 5–7 minutes for BnB+BO. Both gains come from the warm-start mechanism (§5.2). BO-Classic begins from random initial samples, so its convergence depends on whether early samples land in promising regions of the configuration space. BnB pre-filters infeasible configurations and supplies the top BnB-pruned candidates as the BO seed, replacing random initialization with a quality-controlled starting set. The seed's higher quality makes BnB+BO converge faster, and its consistency across trials makes BnB+BO's cost less variable.

7.5 Cluster Simulation Accuracy

§7.4 evaluated the BnB+BO optimizer using simulator outputs. We now check whether the results are consistent with real cluster executions. We use 10 physical bare-metal machines reserved for real-cluster validation. We construct 9 cluster configurations (C1–C9), each comprising 4–7 machines, randomly sampled from the feasible region where both the deadline and cost budget

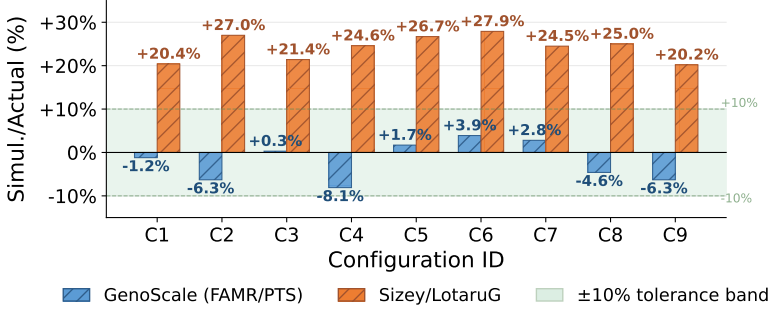


Fig. 12. **Simulation accuracy across 9 configurations.** *GENOSCALE* (FAMR + PTS-Hybrid) achieves 3.9% median and 8.1% worst-case error across all configurations. *Sizey/Lotaru-G* consistently overpredicts by 20.2–27.9%.

are satisfied by simulation. These are the configurations a user would receive from GENOSCALE. For every configuration, we run two simulator pipelines: GENOSCALE’s integrated stack (FAMR + PTS-Hybrid) and a baseline pipeline combining Sizey and Lotaru-G. We then compare each simulator’s predicted makespan against the wall-clock makespan measured on bare-metal.

Fig. 12 shows the results. GENOSCALE’s simulator predicts makespan within 3.9% median absolute error and 8.1% worst-case error across all 9 configurations. The configurations split into two patterns: (1) those dominated by memory-stable tools achieve near-exact predictions (C3 at +0.3%, C5 at +1.7%), and (2) those with concurrent executions of memory-variable tools (PMd, PCWM) show larger underestimation errors (C2 at -6.3%, C4 at -8.1%) because contention amplifies actual makespan beyond what per-task simulation captures.

The baseline simulator pipeline (Sizey + Lotaru-G) often overpredicts makespan by 20.2–27.9%, with every configuration having 20+% error. Two limitations drive this gap. First, Sizey’s point-estimate memory allocation triggers OOM failures on high-variance tools, and the simulator models the resulting retries as reduced effective parallelism. Second, Lotaru-G’s per-task execution time errors (24–30% MAPE) accumulate through DAG dependencies. This gap significantly affects the cluster sizing outputs. The 8.1% worst-case error holds across diverse cluster sizes and tool mixes within the feasible design space, and since BnB+BO selects from this same space, the simulator fidelity directly applies to optimizer-recommended configurations. The baseline’s ~25% error would cause the optimizer to over-provision based on inflated makespan estimates.

7.6 Scaling to Larger Inputs

The per-task evaluation in §7.2 and §7.3 uses raw inputs that are mostly up to 6.6 GB. To test whether FAMR and PTS-Hybrid still provide accurate predictions on larger inputs, we run 30 workflows each at 10, 15, and 18 GB raw inputs. Each run also reads a 17 GB reference genome. We use the Xeon 6240R machine and the same methodology and baselines. PTS-Hybrid is trained only on runs with raw inputs up to 6.6 GB, excluding the few 20 GB runs (§2), and is applied to the larger inputs without retraining. Table 4 summarizes the results, aggregated over the three sizes. Every predictor’s execution-time error varies by less than 1 pp across them.

Execution time. PTS-Hybrid’s MAPE on the larger inputs is 20%, almost unchanged from its 21% on this machine at the original input sizes (Table 4a). While the baselines’ errors decrease on the larger inputs, PTS-Hybrid remains the most accurate predictor at every size. It also has the lowest median error, 15%, the same as on the original inputs (reported in Fig. 8). Per tool, PTS-Hybrid has the lowest MAPE on all but three tools. Its MAPE is higher than Lotaru-G’s by 6 pp on PMd, 2 pp on PCWM, and less than 1 pp on FQ-C. PMd and PCWM are also the two tools where Lotaru-G is more accurate on the original inputs (Table 2).

Table 4. **Prediction accuracy on larger inputs.** Raw inputs of 10, 15, and 18 GB, with each run also reading a 17 GB reference genome. (a) ‘Execution-time prediction (MAPE, %).’ The 0.1–6.6GB column is from Fig. 6 (Xeon 6240R). PTS-Hybrid is applied without retraining. Baselines are averaged over two runs. (b) ‘Memory allocation.’ Bold marks the best per column.

(a) Execution-time prediction (MAPE, %)

Predictor	0.1–6.6 GB	10–18 GB
OnlineP	58	46
Lotaru-A	61	50
Lotaru-G	30	26
PTS-Hybrid	21	20

(b) Memory allocation

Method	Memory Waste	Retry Time	OOM Fail. Rate
FAMR-A	2.12 TB	146Ks	5.9%
FAMR-B	2.19 TB	92.2Ks	4.0%
FAMR-C	2.92 TB	31.9Ks	0.9%
Sizey	2.26 TB	334Ks	14.4%
P95	4.26 TB	74.9Ks	3.7%
WittPer.	4.22 TB	68.3Ks	3.2%
WittReg.	5.60 TB	142Ks	5.9%

Memory. Table 4b reports memory waste, retry time, and OOM failure rate. FAMR reduces retry time by 56–90% relative to Sizey, close to the 61–96% reduction on the original inputs (Table 3). On the larger inputs, Sizey’s memory waste approaches that of FAMR-A and FAMR-B, but its OOM failure rate is 14.4%, vs. 0.9–5.9% for FAMR. As a result, at similar memory waste, Sizey’s retry time is 2.3× and 3.6× that of FAMR-A and FAMR-B, respectively. P95 and WittPer. reduce retry time by allocating high percentiles of past usage, but FAMR-C has both shorter retry time and about 30% less memory waste than either. Every baseline has both higher memory waste and longer retry time than at least one FAMR configuration.

8 Discussion

Generalizability. GENOSCALE targets genomic workflows, but its core components make no assumptions specific to genomic workloads. PTS-Hybrid correlates benchmark scores with processing rates, making it applicable to any CPU-bound workload. FAMR models memory as per-task distributions with temporal dynamics, which is also applicable wherever memory requirements vary across executions. The workflow simulator treats workflows as task DAGs with resource requirements, the standard representation in workflow management systems. These components can be applied to other multi-task scientific workflows that admit the profiling GENOSCALE relies on. The profiling phase (§3.1) runs each tool on a smaller portion of the input (e.g., 5% for genomic workflows), which applies to data-parallel inputs such as sequencing reads or the per-image stages of astronomy mosaics [16, 27], but not to workflows built around a fixed spatial model, such as climate simulation [15, 47] or atomistic materials simulation [31, 55]. Our evaluation covers genomic workflows, and validation on other domains remains future work.

Limitations. GENOSCALE has three limitations. First, our evaluation uses a single workflow family (GATK DNA-seq); validation on RNA-seq, long-read, metagenomic, and single-cell pipelines [18] remains future work. Second, GENOSCALE focuses on CPU and memory prediction. GPU support [42, 46] would require incorporating GPU benchmarks into PTS-Hybrid and extending FAMR to predict GPU memory exhaustion. Third, our experiments run on bare-metal hardware. Virtualized environments may introduce additional variability from noisy neighbors and storage contention that could affect prediction accuracy.

9 Related Work

Bioinformatics Benchmarking. Prior systems characterize performance across hardware and genomic workloads, including containerized single-machine [21, 22], processor-specific tuning [36],

multi-platform [53], GPU heterogeneity [35], cluster-scale HPC [50], pangenomics [28], and integrated workflow management [41]. While these tools focus on standalone measurement, GENOSCALE uses benchmarking output to automatically select heterogeneous cluster configurations under cost and deadline constraints.

Resource Prediction. Workflow execution time prediction either uses analytical models [13, 52] requiring manual development or ML approaches [24, 44, 48] that degrade on hardware outside their training data. Lotaru [6], the SOTA, applies LOESS regression but requires execution history per target machine. PTS-Hybrid instead adapts microbenchmark-based characterization (previously used for HPC procurement [25] and portability analysis [43]) to per-task runtime prediction, achieving 34.1% lower total RMSE without prior workflow data on the target. Memory prediction work mostly produces single-point estimates. Kubernetes VPA [32, 56] uses historical percentiles, Sizey[8] trains multiple regression models and selects per task, and Witt *et al.* [63] use percentile and regression methods. Neural network [19] and reinforcement learning [38] approaches struggle with distribution shift, and none capture temporal dynamics. FAMR instead models memory as per-tool distributions with peak timing, cutting retry time on genomic workflows.

Heterogeneous Cluster Sizing. Selecting cluster resources for scientific workflows is a long-standing research challenge. In genomics, Hummingbird [10] downsamples input to select a single cloud instance, addressing instance selection rather than cluster sizing. For analytics workloads, CherryPick [3] applies BO to select a single VM, with subsequent BO-based works [5, 11] that incorporate profile-based priors and multi-system optimization. HiPerBot [39] extends BO with transfer learning for HPC parameter tuning, and FLASH [49] generalizes this via transfer learning. Ernest [61], Paris [64], Selecta [30], and Arrow [26] leveraged extrapolation, collaborative filtering, storage tiering, and workflow-level configuration on uniform pools. OPTIMUSCLOUD [37] handles heterogeneous VMs for NoSQL workloads that lack DAG dependencies and per-task resource variability. Each of these systems targets a different sizing problem (single instance, homogeneous cluster, or non-DAG workload). Adapting them to our heterogeneous workflow setting would push them outside what they were designed for, so the comparison would not reflect their actual capabilities. Table 5 summarizes their capabilities. GENOSCALE addresses heterogeneous machine selection as a combinatorial problem (2^n subsets) over a pool with varying CPU, memory, and cost, combining BnB+BO with prediction-driven simulation (PTS-Hybrid + FAMR) for rapid *what-if analysis* without expensive trial runs.

10 Conclusion

GENOSCALE enables heterogeneous cluster sizing for genomic workflows by integrating failure-aware memory allocation (FAMR), microbenchmark-driven execution time prediction (PTS-Hybrid), and a BnB+BO optimizer guided by a workflow simulator. On Chameleon Cloud, GENOSCALE reduces task retry time by up to 96%, memory waste by up to 37%, and execution time prediction error by 34.1% over SOTA predictors, and reduces cost by 5.5% over pure BO.

Table 5. **Comparison of cluster resource configuration systems.** ‘Het.’: mixed machine types per cluster; ‘Pred.’: integrated prediction models; ‘\$\$-Opt.’: explicit cost optimization; ‘Min-P.’: minimal profiling.

System	Het.	Pred.	\$\$-Opt.	Min-P.
Ernest [61]	·	✓	✓	·
CherryPick [3]	·	✓	✓	·
Paris [64]	·	✓	·	·
Selecta [30]	·	✓	✓	·
Arrow [26]	·	✓	✓	·
Hummingbird [10]	·	✓	✓	·
HiPerBot [39]	·	·	·	·
OPTIMUSCLOUD [37]	✓	✓	✓	·
Bilal <i>et al.</i> [11]	·	✓	✓	·
PB ³ Opt [5]	·	✓	✓	·
FLASH [49]	·	✓	✓	✓
GENOSCALE	✓	✓	✓	✓

References

- [1] Dong H. Ahn, Xiaohua Zhang, Jeffrey Mast, Stephen Herbein, Francesco Di Natale, Dan Kirshner, Sam Ade Jacobs, Ian Karlin, Daniel J. Milroy, Bronis R. de Supinski, Brian Van Essen, Jonathan E. Allen, and Felice C. Lightstone. 2022. Scalable Composition and Analysis Techniques for Massive Scientific Workflows. In *IEEE International Conference on e-Science (e-Science)*.
- [2] Kursad Albayraktaroglu, Aamer Jaleel, Xue Wu, Manoj Franklin, Bruce L. Jacob, Chau-Wen Tseng, and Donald Yeung. 2005. BioBench: A Benchmark Suite of Bioinformatics Applications. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.
- [3] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. 2017. CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [4] Amazon Web Services. 2026. Amazon EC2 On-Demand Pricing. <https://aws.amazon.com/ec2/pricing/on-demand/>.
- [5] Thais Aparecida Silva Camacho, Vinicius Martins do Rosario, Otavio Oliveira Napoli, and Edson Borin. 2022. PB³Opt: Profile-Based Biased Bayesian Optimization to Select Computing Clusters on the Cloud. *Concurrency and Computation: Practice and Experience* 35, 18 (2022), e7540.
- [6] Jonathan Bader, Fabian Lehmann, Lauritz Thamsen, Ulf Leser, and Odej Kao. 2024. Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures. *Future Generation Computer Systems* 150 (2024), 171–185.
- [7] Jonathan Bader, Ansgar Löffler, Lauritz Thamsen, Björn Scheuermann, and Odej Kao. 2024. KS+: Predicting Workflow Task Memory Usage Over Time. In *IEEE International Conference on e-Science (e-Science)*.
- [8] Jonathan Bader, Fabian Skalski, Fabian Lehmann, Dominik Scheinert, Jonathan Will, Lauritz Thamsen, and Odej Kao. 2024. Sizely: Memory-Efficient Execution of Scientific Workflow Tasks. In *IEEE International Conference on Cluster Computing (CLUSTER)*.
- [9] Jonathan Bader, Kathleen West, Sören Becker, Svetlana Kulagina, Fabian Lehmann, Lauritz Thamsen, Henning Meyerhenke, and Odej Kao. 2025. Predicting the Performance of Scientific Workflow Tasks for Cluster Resource Management: An Overview of the State of the Art. *CoRR* abs/2504.20867 (2025).
- [10] Amir Bahmani, Ziye Xing, Vandhana Krishnan, Utsab Ray, Frank Mueller, Amir Alavi, Philip S. Tsao, Michael P. Snyder, and Cuiping Pan. 2021. Hummingbird: Efficient Performance Prediction for Executing Genomic Applications in the Cloud. *Bioinformatics* 37, 17 (2021), 2537–2543.
- [11] Muhammad Bilal, Marco Canini, and Rodrigo Rodrigues. 2020. Finding the Right Cloud Configuration for Analytics Clusters. In *ACM Symposium on Cloud Computing (SoCC)*.
- [12] Broad Institute. 2019. Picard toolkit. <https://broadinstitute.github.io/picard/>. Broad Institute, GitHub Repository.
- [13] Alexandru Calotoiu, Torsten Hoefler, Marius Poke, and Felix Wolf. 2013. Using Automated Performance Modeling to Find Scalability Bugs in Complex Codes. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [14] Rafael Ferreira da Silva, Gideon Juve, Mats Rynge, Ewa Deelman, and Miron Livny. 2015. Online Task Resource Consumption Prediction for Scientific Workflows. *Parallel Processing Letters* 25, 03 (2015), 1541003.
- [15] G. Danabasoglu, J.-F. Lamarque, J. Bacmeister, et al. 2020. The Community Earth System Model Version 2 (CESM2). *Journal of Advances in Modeling Earth Systems* 12, 2 (2020), e2019MS001916.
- [16] Ewa Deelman, Gurmeet Singh, Miron Livny, Bruce Berriman, and John Good. 2008. The Cost of Doing Science on the Cloud: The Montage Example. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [17] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuang-Ching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. 2019. The Design and Operation of CloudLab. In *USENIX Annual Technical Conference (USENIX ATC)*.
- [18] Philip A. Ewels, Alexander Peltzer, Sven Fillinger, Harshil Patel, Johannes Alneberg, Andreas Wilm, Maxime Ulysse Garcia, Paolo Di Tommaso, and Sven NahnSEN. 2020. The nf-core framework for community-curated bioinformatics pipelines. *Nature Biotechnology* 38, 3 (2020), 276–278. <https://doi.org/10.1038/s41587-020-0439-x>
- [19] Wei Gao, Zhisheng Ye, Peng Sun, Yonggang Wen, and Tianwei Zhang. 2020. Estimating GPU Memory Consumption of Deep Learning Models. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.
- [20] Google Cloud. 2026. Google Cloud Compute Engine Pricing. <https://cloud.google.com/compute/all-pricing>.
- [21] Maximilian Hanussek, Felix Bartusch, and Jens Krüger. 2020. BOOTABLE: Bioinformatics benchmark tool suite for applications and hardware. *Future Generation Computer Systems* 102 (2020).
- [22] Maximilian Hanussek, Felix Bartusch, and Jens Krüger. 2021. Performance and scaling behavior of bioinformatic applications in virtualization environments to create awareness for the efficient use of compute resources. *PLoS*

- Computational Biology* 17, 7 (2021).
- [23] Herodotos Herodotou, Fei Dong, and Shivnath Babu. 2011. No One (Cluster) Size Fits All: Automatic Cluster Sizing for Data-Intensive Analytics. In *ACM Symposium on Cloud Computing (SOCC)*.
 - [24] Muhammad Hilman, Maria Rodriguez, and Rajkumar Buyya. 2018. Task Runtime Prediction in Scientific Workflows Using an Online Incremental Learning Approach. In *IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*.
 - [25] Torsten Hoefler and Roberto Belli. 2015. Scientific Benchmarking of Parallel Computing Systems. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
 - [26] Chin-Jung Hsu, Vivek Nair, Vincent W. Freeh, and Tim Menzies. 2018. Arrow: Low-Level Augmented Bayesian Optimization for Finding the Best Cloud VM. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*.
 - [27] Gideon Juve, Ann Chervenak, Ewa Deelman, Shishir Bharathi, Gaurang Mehta, and Karan Vahi. 2013. Characterizing and Profiling Scientific Workflows. *Future Generation Computer Systems* 29, 3 (2013), 682–692.
 - [28] Noah Kaplan, Jan-Niklas Schmelzle, Yufeng Gu, Erik Garrison, Christopher Batten, and Reetuparna Das. 2025. PangenomicsBench: A Benchmark Suite and Characterization of Pangenomics. In *IEEE International Symposium on Workload Characterization (IISWC)*.
 - [29] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. 2020. Lessons Learned from the Chameleon Testbed. In *2020 USENIX Annual Technical Conference (USENIX ATC)*.
 - [30] Ana Klimovic, Heiner Litz, and Christos Kozyrakis. 2018. Selecta: Heterogeneous Cloud Storage Configuration for Data Analytics. In *USENIX Annual Technical Conference (ATC)*.
 - [31] G. Kresse and J. Furthmüller. 1996. Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set. *Physical Review B* 54, 16 (1996), 11169–11186.
 - [32] Kubernetes Authors. 2026. Vertical Pod Autoscaler. <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>
 - [33] Jeremy Leipzig. 2017. A Review of Bioinformatic Pipeline Frameworks. *Briefings in Bioinformatics* 18, 3 (2017), 530–536.
 - [34] Heng Li and Richard Durbin. 2010. Fast and accurate long-read alignment with Burrows–Wheeler transform. *Bioinformatics* 26, 5 (2010), 589–595.
 - [35] Zhuren Liu, Shouzhe Zhang, Justin Garrigus, and Hui Zhao. 2023. Genomics-GPU: A Benchmark Suite for GPU-accelerated Genome Analysis. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.
 - [36] Lorién López-Villellas, Rubén Langarita-Benítez, Asaf Badouh, Víctor Soria Pardos, Quim Aguado-Puig, Guillem López-Paradís, Max Doblas, Javier Setoain, Chulho Kim, Makoto Ono, Adrià Armejach, Santiago Marco-Sola, Jesús Alastruey-Benedé, Pablo Ibáñez, and Miquel Moretó. 2024. GenArchBench: A genomics benchmark suite for arm HPC processors. *Future Generation Computer Systems* 157 (2024).
 - [37] Ashraf Mahgoub, Alexander Michaelson Medoff, Rakesh Kumar, Subrata Mitra, Ana Klimovic, Somali Chaterji, and Saurabh Bagchi. 2020. OPTIMUSCLOUD: Heterogeneous Configuration Optimization for Distributed Databases in the Cloud. In *USENIX Annual Technical Conference (USENIX ATC)*.
 - [38] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrisnan, Zili Meng, and Mohammad Alizadeh. 2019. Learning Scheduling Algorithms for Data Processing Clusters. In *ACM SIGCOMM Conference (SIGCOMM)*.
 - [39] Harshitha Menon, Abhinav Bhatele, and Todd Gamblin. 2020. Auto-Tuning Parameter Choices in HPC Applications Using Bayesian Optimization. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*.
 - [40] Felix Mölder, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twarber, Allen Kanber, Simon Wilber, Matthew Hemberg, Altuna Akalin, Nigel Delaney, Luca Costanzo, and Johannes Köster. 2021. Sustainable Data Analysis with Snakemake. *FI1000Research* 10 (2021), 33.
 - [41] Maria Luiza Mondelli, Thiago Magalhães, Guilherme Loss, Michael Wilde, Ian T. Foster, Marta Mattoso, Daniel S. Katz, Helio J. C. Barbosa, Ana Tereza Ribeiro de Vasconcelos, Kary A. C. S. Ocaña, and Luiz M. R. Gadelha Jr. 2018. BioWorkbench: A High-Performance Framework for Managing and Analyzing Bioinformatics Experiments. *CoRR* abs/1801.03915 (2018).
 - [42] Kyle A. O’Connell, Zelaikha B. Yosufzai, Ross Campbell, Collin J. Lobb, Haley T. Engelken, Laura M. Gorrell, Thad B. Carlson, Josh J. Catana, Dina Mikdadi, Vivien R. Bonazzi, and Juergen A. Klenk. 2023. Accelerating genomic workflows using NVIDIA Parabricks. *BMC Bioinformatics* 24, 1 (2023), 221.
 - [43] Simon J. Pennycook, Jason D. Sewall, and Victor W. Lee. 2016. A Metric for Performance Portability. *CoRR* abs/1611.07409 (2016).
 - [44] Tuan Pham, Juan J. Durillo, and Thomas Fahringer. 2020. Predicting Workflow Task Execution Time in the Cloud Using a Two-Stage Machine Learning Approach. *IEEE Transactions on Cloud Computing* 8, 1 (2020), 256–268.

- [45] Phoronix Media. 2026. Phoronix Test Suite: Open-Source Automated Benchmarking. <https://www.phoronix-test-suite.com/>
- [46] Ryan Poplin, Pi-Chuan Chang, David Alexander, et al. 2018. A universal SNP and small-indel variant caller using deep neural networks. *Nature Biotechnology* 36 (2018), 983–987.
- [47] Jordan G. Powers, Joseph B. Klemp, William C. Skamarock, et al. 2017. The Weather Research and Forecasting Model: Overview, System Efforts, and Future Directions. *Bulletin of the American Meteorological Society* 98, 8 (2017), 1717–1737.
- [48] Sarunya Pumma, Wu-chun Feng, Phond Phunchongharn, Sylvain Chapeland, and Tiranee Achalakul. 2017. A runtime estimation framework for ALICE. *Future Generation Computer Systems* 72 (2017), 65–77.
- [49] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Basar, and Ravi K. Iyer. 2024. FLASH: Fast Model Adaptation in ML-Centric Cloud Platforms. In *The Seventh Annual Conference on Machine Learning and Systems (MLSys)*.
- [50] Seungwoo Rho, Geunchul Park, Jieun Choi, and Chan-Yeol Park. 2021. Development of benchmark automation suite and evaluation of various high-performance computing systems. *Cluster Computing* 24, 1 (2021).
- [51] Michel J. F. Rosa, Célia Ghedini Ralha, Maristela Holanda, and Aletéia P. F. Araújo. 2021. Computational resource and cost prediction service for scientific workflows in federated clouds. *Future Generation Computing Systems* 125 (2021), 844–858.
- [52] Allan Snaveley, Laura Carrington, Nicole Wolter, Jesús Labarta, Rosa Badia, and Avi Purkayastha. 2002. A Framework for Performance Modeling and Prediction. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [53] Arun Subramanian, Yufeng Gu, Timothy Dunn, Somnath Paul, Md. Vasimuddin, Sanchit Misra, David T. Blaauw, Satish Narayanasamy, and Reetuparna Das. 2021. GenomicsBench: A Benchmark Suite for Genomics. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.
- [54] sysstat project. 2026. sysstat: System Performance Tools. <https://sysstat.github.io/>.
- [55] Aidan P. Thompson, Hasan Metin Aktulga, Richard Berger, Dan S. Bolintineanu, W. Michael Brown, Paul S. Crozier, Pieter J. In 't Veld, Axel Kohlmeyer, Stan G. Moore, Trung Dac Nguyen, Ray Shan, Mark J. Stevens, Julien Tranchida, Christian Trott, and Steven J. Plimpton. 2022. LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications* 271 (2022), 108171.
- [56] Muhammad Tirmazi, Adam Barker, Nan Deng, Md E. Haque, Zhijing Gene Qin, Steven Hand, Mor Harchol-Balter, and John Wilkes. 2020. Borg: The Next Generation. In *European Conference on Computer Systems (EuroSys)*.
- [57] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. Nextflow Enables Reproducible Computational Workflows. *Nature Biotechnology* 35, 4 (2017), 316–319.
- [58] Michael Hao Tong, Robert L. Grossman, and Haryadi S. Gunawi. 2021. Experiences in Managing the Performance and Reliability of a Large-Scale Genomics Cloud Platform. In *USENIX Annual Technical Conference (USENIX ATC)*.
- [59] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel Distributed Systems* 13, 3 (2002), 260–274.
- [60] Geraldine A. Van der Auwera, Mauricio O. Carneiro, Christopher Hartl, Ryan Poplin, Guillermo del Angel, Ami Levy-Moonshine, Tadeusz Jordan, Khalid Shakir, David Roazen, Joel Thibault, Eric Banks, Kiran V. Garimella, David Altshuler, Stacey Gabriel, and Mark A. DePristo. 2013. From FastQ Data to High-Confidence Variant Calls: The Genome Analysis Toolkit Best Practices Pipeline. *Current Protocols in Bioinformatics* 43 (2013), 11.10.1–11.10.33.
- [61] Shivaram Venkataraman, Zongheng Yang, Michael J. Franklin, Benjamin Recht, and Ion Stoica. 2016. Ernest: Efficient Performance Prediction for Large-Scale Advanced Analytics. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [62] Kris A. Wetterstrand. 2023. DNA Sequencing Costs: Data from the NHGRI Genome Sequencing Program (GSP). <https://www.genome.gov/about-genomics/fact-sheets/DNA-Sequencing-Costs-Data>.
- [63] Carl Witt, Dennis Wagner, and Ulf Leser. 2019. Feedback-Based Resource Allocation for Batch Scheduling of Scientific Workflows. In *International Conference on High Performance Computing & Simulation (HPCS)*.
- [64] Neeraja J. Yadwadkar, Bharath Hariharan, Joseph E. Gonzalez, Burton Smith, and Randy H. Katz. 2017. Selecting the Best VM across Multiple Public Clouds: A Data-Driven Performance Modeling Approach. In *ACM Symposium on Cloud Computing (SoCC)*.